Part 3: Strategy and Observer across four languages

Course: 23CSE455 Design Patterns **Team:** B14 **Application studied:** Next.js Blog CMS at baseline commit `90ec656`

Scope: Two patterns, with examples in Python, JavaScript, Java and C++

1. What we compared

Part 1 confirmed an LLM provider streaming Strategy and an auth-state Observer in the existing application. RBAC was also counted in Part 1, but the team selected the two behavioral patterns for Part 3. These are small, runnable teaching examples. They do not call real LLM providers or Supabase, and they are not ports of the full Next.js app.

Govind's Python and JavaScript examples share one JSON input/output contract and 26 saved cases. Soorya's Java and C++ examples share a different, smaller contract with four cases per pattern. Each pair has passing tests. One common four-language test set has not been run, so the results do not prove four-way behavioral equivalence.

2. Where the patterns came from

Strategy

In `code/features/ai-assistant/llmService.ts`, `sendMessage` chooses `streamClaude`, `streamOpenAI` or `streamGemini` using `params.provider`. Those functions have the same input and output shape. The original app uses a function-based contract and an `if` chain; it does not declare a Strategy interface.

Govind's Python and JavaScript examples keep function-based strategies. Their `run` function selects a provider-specific extractor for saved sample events. Soorya's Java example uses an `LlmStrategy` interface and classes; the C++ example uses an abstract base class and concrete classes. The Java/C++ providers return fixed text pieces for repeatable tests. None of the four examples makes a network request.

Observer

In `code/features/auth/context/AuthProvider.tsx`, `supabase.auth` supplies the subject. `AuthProvider` subscribes with `onAuthStateChange`, updates session/user/profile state when notified, and unsubscribes during cleanup. `AuthContext` makes that state available to children. The subject belongs to Supabase, not to the original app's own classes.

Govind's Python and JavaScript examples use an in-memory `AuthSubject` and an `AuthStateObserver` that records user IDs. Soorya's Java and C++ examples use an event source and recording listeners. All four examples demonstrate subscribe, notification and unsubscribe without connecting to Supabase.

3. Relationship to Part 2

The cumulative Part 2 application changes are **CH01 related articles**, **CH02 content moderation** and **CH03 premium content**. The report's `code/` folder still contains the original application snapshot. The evolved feature code is in a separate application branch recorded in `../Part2/` and the team report. The original Strategy and Observer collaborations remain present in that branch. The related-article ranker is one fixed algorithm, and moderation and premium access add rules without establishing a new GoF pattern instance.

Part 2 also records three **separate refactoring experiments**: provider inference in `generateBlogPost()`, publication title/content validation helpers, and shared generated-post JSON parsing. These are not the feature changes CH01 to CH03. The baseline `code/features/ai-assistant/llmService.ts` copy does not contain the provider-inference or JSON-parser helper from those experiment branches. Part 3 uses the verified baseline pattern mapping and does not claim to benchmark the evolved source.

4. Tests and results

Govind's saved inputs are in `llm_provider_streaming/fixtures.json` and `auth_state_observer/fixtures.json`. The Python and JavaScript test runners read those same files and compare both the result and the unchanged input. `verify_cross_language.py` also runs each program as a separate process and checks both outputs against the same expected JSON.

Pattern	Python	JavaScript	Shared process check
Strategy	11/11 passed	11/11 passed	11/11 matched
Observer	15/15 passed	15/15 passed	15/15 matched

The combined Python/JavaScript process check passed **26/26**. Its log and the four language-specific logs are in `evidence/tests/`.

Soorya's separate test specification is `test-cases/common_test_cases.csv`. It covers provider selection and rejection for Strategy, then notification, unsubscribe and multiple listeners for Observer. The saved compile/test output in `evidence/test_results.txt` records **4/4** cases per pattern in Java and **4/4** per pattern in C++.

The Python/JavaScript inputs are provider event JSON and auth event JSON. The Java/C++ inputs are prompt/provider strings and direct event/session calls. These are different workloads; their pass counts cannot be added into one shared four-language result.

5. Code size, complexity and timing

Govind measured production source with Lizard 1.24.1. His process timings used one warm-up and five measured runs on Windows. They include startup and JSON file input/output. Raw values are in `evidence/static_metrics.json` and `evidence/timings/process_timings.json`; the four summary rows are in `../data/cross_language.csv`.

Soorya counted production lines and performed a documented manual McCabe audit. His Java/C++ timings used one warm-up and five runs in a different environment, including the test harness but excluding compilation. Raw values and the counting rule are in `evidence/timing_raw.csv`, `evidence/complexity_audit.csv` and `evidence/complexity_audit.md`. His four rows remain in `cross_language.csv`.

Pattern	Language	Production lines	Max function CC	Median process time (ms)
Strategy	Python	53	8	96.870
Strategy	JavaScript	47	9	57.485
Observer	Python	74	21	100.344
Observer	JavaScript	62	20	57.960
Strategy	Java	33	4	36.116
Strategy	C++	35	4	1.115
Observer	Java	24	2	42.995
Observer	C++	30	2	1.070

The methods and workloads differ, so these numbers do not rank the languages. In Govind's Observer examples, input validation accounts for most of the highest function complexity. The original app's pattern roles are clearer than a timing comparison of these small programs.

6. Files and remaining checks

- `llm_provider_streaming/` and `auth_state_observer/`: Govind's Python/JavaScript source, shared fixtures, tests, diagrams and run instructions.
- `strategy/` and `observer/`: Soorya's Java/C++ source and tests.
- `README.md`: commands for each language and links to evidence.
- `llm/govind_prompt.txt`: Govind's saved prompt record. `llm/verification.md` explains source and test checks. The partial `llm/llm.csv` row marks the raw output, model name and date unavailable.
- `llm/part3_prompt.txt`: prompt record for the Java/C++ working phase.

For a final four-language claim, the team still needs to agree on one input/output specification, run its cases in all four languages, and then reconcile the two CSVs and measurement boundaries. Any missing LLM output or model information must come from actual saved history. This report does not infer it.